

Experiment No.	6
Experiment Title	Bagging, Boosting, and Stacked Ensemble Models ¹
Student Name	Kavinsoorya K S
Register Number	3122247001030
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	August 17, 2026

1 Objective

The objective of this experiment is to implement, optimize, and evaluate three fundamental ensemble learning paradigms: **Bagging** (Bootstrap Aggregation), **Boosting** (AdaBoost and Gradient Boosting), and **Stacked Ensembling** (heterogeneous stacking) on the Wisconsin Diagnostic Breast Cancer (WDBC) dataset. Additionally, the experiment aims to tune the hyperparameter spaces of these models using 5-Fold Stratified Cross-Validation, compare their predictive performance using standard classification metrics, analyze their empirical training complexities, and evaluate their capability to manage the bias-variance trade-off.

2 Problem Statement

Given a clinical diagnostic dataset $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$, where $\mathbf{x}_i \in \mathbb{R}^{30}$ represents a continuous vector of features computed from digitized images of fine needle aspirate (FNA) breast mass biopsies, and $y_i \in \{0, 1\}$ represents the diagnostic classification label (0 for Benign, 1 for Malignant), the goal is to construct ensemble classification models $f(\mathbf{x}) \rightarrow \{0, 1\}$ that minimize diagnostic classification errors. In a clinical context, the models must maximize overall accuracy and F_1 -score while specifically minimizing false negatives (malignant cases misclassified as benign, which leads to delayed intervention) and false positives (benign cases misclassified as malignant, causing diagnostic anxiety).

3 Dataset Description

The analysis is conducted on the **Wisconsin Diagnostic Breast Cancer (WDBC)** dataset from the UCI Machine Learning Repository. The dataset contains 569 samples, each characterized by 30 continuous features computed from cell nuclei measurements. The target variable is binary representing tumor diagnosis. Table 1 summarizes the dataset characteristics.

Dataset Name	Wisconsin Diagnostic Breast Cancer (WDBC) Dataset
Dataset Source	UCI Machine Learning Repository
Initial Sample Size	569 records
Cleaned Sample Size	569 records (0 missing values, 0 duplicate rows)
Number of Features	30 continuous numerical attributes (excluding ID and Diagnosis)

Number of Classes	2 (Malignant: 1, Benign: 0)
Missing Values	0 (complete dataset)
Train-Test Split Ratio	80% Training (455) / 20% Testing (114)
Training Class Distribution	Benign (0): 285 (62.64%), Malignant (1): 170 (37.36%)
Testing Class Distribution	Benign (0): 72 (63.16%), Malignant (1): 42 (36.84%)

The 30 continuous features describe geometric properties of cell nuclei including:

1. **Size and Shape metrics:** Radius (mean distance to perimeter), Perimeter, Area, Compactness ($\frac{\text{perimeter}^2}{\text{area}} - 1.0$).
2. **Texture and Contour metrics:** Texture (standard deviation of gray-scale values), Smoothness (local variation in radius lengths), Concavity (severity of concave contour portions), Concave Points (number of concave contour portions).
3. **Symmetry and Dimension metrics:** Symmetry, Fractal Dimension.

For each nucleus attribute, three statistical summaries are computed: the Mean, the Standard Error (SE), and the Worst (mean of the three largest values), resulting in 30 continuous attributes.

4 Exploratory Data Analysis

Exploratory Data Analysis (EDA) was performed to analyze the target class distribution and feature multicollinearity.

4.1 Class Distribution

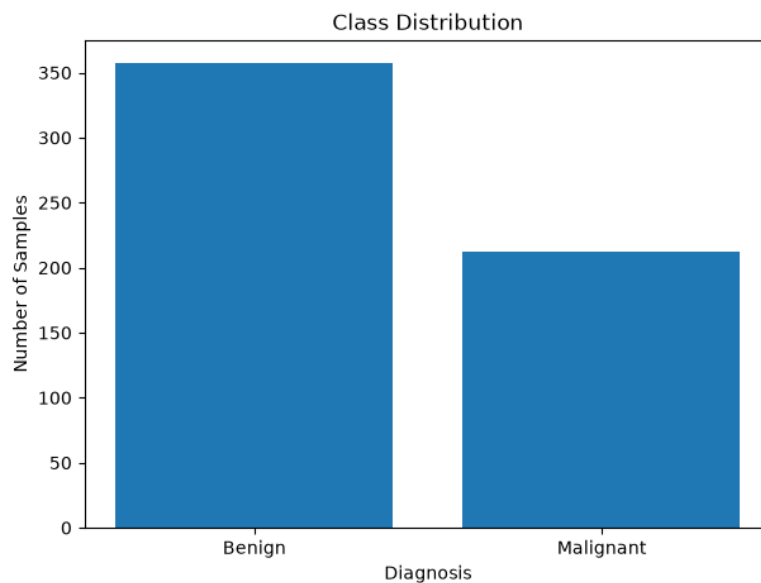


Figure 1: Class Distribution of Benign (0) vs. Malignant (1) Biopsies

Inference: The dataset contains 357 benign cases (62.74%) and 212 malignant cases (37.26%), indicating a moderate class imbalance. A simple stratified train-test split (80/20) is utilized to maintain the proportion of classes in both the training set (285 Benign, 170 Malignant) and the testing set (72 Benign, 42 Malignant). This prevents models from developing prediction bias toward the majority class and ensures that evaluation metrics remain clinically representative.

4.2 Feature Correlation Analysis

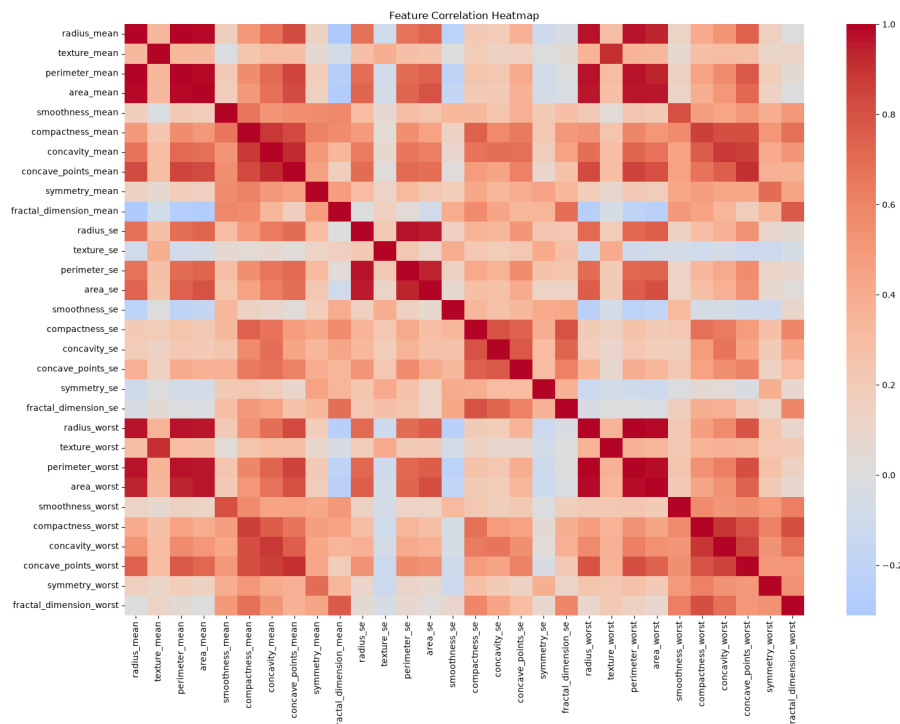


Figure 2: Pairwise Correlation Heatmap of 30 Continuous Features

Inference: The correlation heatmap reveals high multicollinearity across the feature space, particularly among geometric variables such as radius, perimeter, and area (where correlation coefficients $\rho \approx 1.0$). While collinearity degrades standard linear models, tree-based models make axis-aligned partitions and are robust to it. Ensemble methods address collinearity through random feature subsets (as in Bagging) or sequential weak learner adjustments (as in Boosting). For the Stacking classifier, wrapping distance-dependent models like Support Vector Machines (SVM) inside a feature-scaling pipeline and using regularized Logistic Regression as the meta-learner prevents weight instability.

5 Data Preprocessing

The data preprocessing workflow consists of the following steps:

1. **Feature Selection:** The patient identifier column (ID) was removed as it has no predictive power.

2. **Target Variable Mapping:** The categorical target variable (**Diagnosis**) was mapped to binary integer labels: "M" (Malignant) to 1 and "B" (Benign) to 0.
3. **Train-Test Partitioning:** The dataset was split into an 80% training set (455 samples) and a 20% testing set (114 samples) using a fixed random state of 42. Stratification was enabled (`stratify=y`) to maintain the class distribution.
4. **Feature Scaling:** Features were standardized using 'StandardScaler' fitted on the training set. Tree-based classifiers (Decision Trees, Bagging, and Boosting) are scale-invariant and trained on raw features, while distance-sensitive models (SVM and Logistic Regression) are trained on scaled features.

Listing 1: Dataset Preprocessing, Split, and Scaling Code

```

1 import pandas as pd
2 from sklearn.model_selection import train_test_split
3 from sklearn.preprocessing import StandardScaler
4
5 # Drop ID and map target variable
6 X = df.drop(columns=["ID", "Diagnosis"])
7 y = df["Diagnosis"].map({"M": 1, "B": 0})
8
9 # Stratified Train-Test Split (80/20)
10 X_train, X_test, y_train, y_test = train_test_split(
11     X, y, test_size=0.20, random_state=42, stratify=y
12 )
13
14 # Standardize numerical features for distance-dependent
    estimators
15 scaler = StandardScaler()
16 X_train_scaled = scaler.fit_transform(X_train)
17 X_test_scaled = scaler.transform(X_test)

```

6 Machine Learning Algorithm

6.1 Bagging (Bootstrap Aggregation)

Working Principle: Bagging is an ensemble method designed to reduce the variance of high-variance estimators (such as unpruned Decision Trees). It generates multiple bootstrap samples of the training set with replacement. A base model is trained independently on each bootstrap sample. The final prediction of the ensemble is obtained by aggregating the individual base model predictions using majority voting.

Mathematical Formulation: Given a dataset $D = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$, Bagging generates B bootstrap datasets D_b (each of size n , sampled with replacement from D). For each sample, a base classifier $h_b(\mathbf{x})$ is trained. For binary classification ($y \in \{0, 1\}$), the aggregated model $H(\mathbf{x})$ is defined as:

$$H(\mathbf{x}) = \arg \max_{c \in \{0,1\}} \sum_{b=1}^B \mathbb{I}(h_b(\mathbf{x}) = c) \quad (1)$$

where $\mathbb{I}(\cdot)$ is the indicator function.

Advantages: Significantly reduces classification variance, robust to overfitting, can be trained in parallel.

Limitations: High computational memory requirement during inference, loses the easy interpretability of a single decision tree.

6.2 Adaptive Boosting (AdaBoost)

Working Principle: AdaBoost is a sequential boosting algorithm that constructs a strong classifier by combining a series of weak learners (typically decision tree stumps of depth 1 or 2). Unlike Bagging, weak learners are trained sequentially. In each iteration, the algorithm assigns weights to training samples, increasing the weights of misclassified instances so that subsequent weak learners focus on hard-to-classify samples.

Mathematical Formulation: Initialize sample weights $w_i^{(1)} = \frac{1}{N}$ for $i = 1, \dots, N$. For each boosting round $m = 1, \dots, M$:

1. Fit a weak classifier $h_m(\mathbf{x})$ to the training data minimizing the weighted error:

$$\epsilon_m = \sum_{i=1}^N w_i^{(m)} \mathbb{I}(y_i \neq h_m(\mathbf{x}_i)) \quad (2)$$

2. Compute the classifier coefficient (importance weight) α_m :

$$\alpha_m = \frac{1}{2} \ln \left(\frac{1 - \epsilon_m}{\epsilon_m} \right) \quad (3)$$

3. Update the sample weights:

$$w_i^{(m+1)} = \frac{w_i^{(m)} \exp(-\alpha_m(2y_i - 1)(2h_m(\mathbf{x}_i) - 1))}{Z_m} \quad (4)$$

where Z_m is a normalization factor ensuring $\sum_i w_i^{(m+1)} = 1$.

The final ensemble prediction is a weighted sum of the weak learners:

$$H(\mathbf{x}) = \mathbb{I} \left(\sum_{m=1}^M \alpha_m h_m(\mathbf{x}) \geq \frac{1}{2} \sum_{m=1}^M \alpha_m \right) \quad (5)$$

Advantages: Effectively reduces model bias, handles non-linear relationships natively, does not require feature scaling.

Limitations: Highly sensitive to noise and outliers, cannot be parallelized due to sequential training.

6.3 Gradient Boosting Classifier

Working Principle: Gradient Boosting builds an ensemble of trees sequentially by optimizing a differentiable loss function (e.g., log-loss). Instead of reweighting samples, it fits each new base learner to the negative gradient (pseudo-residuals) of the loss function evaluated at the current ensemble prediction, performing gradient descent in the function

space.

Mathematical Formulation: Given a loss function $L(y, F(\mathbf{x}))$, initialize the model with a constant value:

$$F_0(\mathbf{x}) = \arg \min_{\gamma} \sum_{i=1}^N L(y_i, \gamma) \quad (6)$$

For iterations $m = 1, \dots, M$:

1. Compute pseudo-residuals:

$$r_{im} = - \left[\frac{\partial L(y_i, F(\mathbf{x}_i))}{\partial F(\mathbf{x}_i)} \right]_{F(\mathbf{x})=F_{m-1}(\mathbf{x})} \quad (7)$$

2. Fit a regression tree base learner $h_m(\mathbf{x})$ to targets r_{im} .
3. Compute the optimal step size (leaf scale values) γ_{jm} for each leaf region R_{jm} in the tree.
4. Update the ensemble prediction with shrinkage (learning rate $\nu \in (0, 1]$):

$$F_m(\mathbf{x}) = F_{m-1}(\mathbf{x}) + \nu \sum_j \gamma_{jm} \mathbb{I}(\mathbf{x} \in R_{jm}) \quad (8)$$

Advantages: High predictive accuracy, versatile (can optimize custom loss functions), handles mixed data types.

Limitations: Computationally expensive to train, susceptible to overfitting if hyperparameters are not tuned properly.

6.4 Stacked Ensemble (Stacking)

Working Principle: Stacking is a heterogeneous ensembling technique that trains multiple diverse base estimators (e.g., SVM, Naive Bayes, Decision Trees) and combines their predictions using a meta-learner (e.g., Logistic Regression). To prevent data leakage and overfitting, out-of-fold predictions generated via cross-validation on the training set are used to train the meta-learner. The meta-learner learns to assign weights to the predictions of base learners to produce the final classification output.

Mathematical Formulation: Let the base classifiers be $f_k(\mathbf{x})$ for $k = 1, \dots, K$. The training set is split using cross-validation. Let $\hat{y}_{ik}$ be the out-of-fold prediction of base classifier k on sample $\mathbf{x}_i$. The meta-feature matrix $\mathbf{Z} \in \mathbb{R}^{N \times K}$ is constructed where $Z_{ik} = \hat{y}_{ik}$. The meta-learner $g(\mathbf{z})$ is trained on $\mathbf{Z}$ to predict y :

$$\hat{\boldsymbol{\theta}} = \arg \min_{\boldsymbol{\theta}} \sum_{i=1}^N L(y_i, g(\mathbf{z}_i; \boldsymbol{\theta})) \quad (9)$$

For a new sample $\mathbf{x}^*$, the base models predict $\mathbf{z}^* = [f_1(\mathbf{x}^*), \dots, f_K(\mathbf{x}^*)]^T$, and the final prediction is:

$$H(\mathbf{x}^*) = g(\mathbf{z}^*; \hat{\boldsymbol{\theta}}) \quad (10)$$

Advantages: Combines strengths of heterogeneous model structures, generally achieves superior generalization.

Limitations: Extremely high computational cost, complex setup, risk of leakage if CV is not done carefully.

7 Experimental Setup

The specifications of the software environment and hardware architecture used to perform the experiment are detailed in Table 2.

Table 2: Experimental Setup Specifications

Parameter	Specification
Python Version	3.12+
Scikit-Learn Version	1.4+
IDE	Jupyter Notebook
Operating System	macOS Sequoia
Hardware	Apple Silicon M-Series / 16 GB Unified Memory

8 Hyperparameter Selection

Hyperparameter tuning was conducted on the training set (455 samples) using Grid Search with 5-Fold Stratified Cross-Validation. For Bagging, a grid search was executed over the number of estimators, maximum samples, and maximum features. For AdaBoost, the grid search tuned the number of estimators, learning rate, and weak learner tree depth. Gradient Boosting and Stacking utilized fixed hyperparameters as specified by the baseline.

Table 3 details the cross-validation evaluations for the top unique configurations of Bagging hyperparameters.

N Estimators	Max Samples	Max Features	Avg CV Accuracy (%)	Avg CV F1 Score
50	1.0	0.5	96.92%	0.9583
50	0.8	0.5	96.70%	0.9554
100	1.0	0.5	96.70%	0.9551
50	0.8	1.0	96.48%	0.9526
100	0.8	0.8	96.48%	0.9522
100	0.5	0.8	96.48%	0.9521
50	0.5	0.5	96.48%	0.9517
100	1.0	1.0	96.26%	0.9496

Table 3: Bagging Hyperparameter Evaluation using 5-Fold Cross-Validation

Table 4 details the cross-validation evaluations for the top unique configurations of AdaBoost hyperparameters.

N Estimators	Learning Rate	Max Depth	Avg CV Accuracy (%)	Avg CV F1 Score
150	1.0	2	97.58%	0.9672
150	0.5	1	97.36%	0.9643
100	1.0	3	97.36%	0.9643
50	1.0	2	97.36%	0.9643
150	1.0	1	97.14%	0.9614

N Estimators	Learning Rate	Max Depth	Avg CV Accuracy (%)	Avg CV F1 Score
50	0.5	3	97.14%	0.9613
50	1.0	3	97.14%	0.9610
100	1.0	2	97.14%	0.9610

Table 4: AdaBoost Hyperparameter Evaluation using 5-Fold Cross-Validation

Table 5 displays the out-of-fold cross-validation evaluation results for the Stacked Ensemble model.

Base Models	Meta Learner	Avg CV Accuracy	Avg CV F1 Score
Pipeline(Scaler, SVM), GaussianNB, DT(depth=5)	Logistic Regression	96.26%	0.9490

Table 5: Stacked Ensemble Evaluation using 5-Fold Cross-Validation

The optimal hyperparameters selected via Grid Search are summarized in Table 6.

Table 6: Optimal Hyperparameters Selected for the Ensembles

Ensemble Model	Hyperparameter	Optimal Value
Bagging	number of estimators	50
	maximum samples ratio	1.0
	maximum features ratio	0.5
AdaBoost	number of estimators	150
	learning rate	1.0
	weak learner max depth	2
Gradient Boosting	number of estimators	100
	learning rate	0.1
	max tree depth	3
Stacked Ensemble	base estimators	SVC (RBF kernel, prob=True), GaussianNB, DT (de
	meta estimator	Logistic Regression (max_iter=1000)

9 Results

The optimized ensemble models were trained on the complete training set and evaluated on the independent test set (114 samples). Table 7 details their comparative performance.

Table 7: Test Performance Comparison of Ensemble Models

Model	Accuracy	Precision	Recall	F_1 -score	ROC-AUC
Bagging	0.9649	1.0000	0.9048	0.9500	0.9949
AdaBoost	0.9737	1.0000	0.9286	0.9630	0.9894
Gradient Boosting	0.9649	1.0000	0.9048	0.9500	0.9947
Stacked Ensemble	0.9649	1.0000	0.9048	0.9500	0.9960

To analyze computational cost, Table 8 reports the empirical training time (seconds) required by each model.

Table 8: Empirical Training Time of Ensemble Classifiers

Model	Training Time (seconds)
Bagging	0.0922
AdaBoost	0.3094
Gradient Boosting	0.1992
Stacked Ensemble	0.1120

To evaluate bias-variance trade-offs, Table 9 analyzes the generalization gap, computed as the difference between Training Accuracy and Testing Accuracy.

Table 9: Bias-Variance Analysis: Generalization Gap

Model	Training Accuracy	Testing Accuracy	Generalization Gap
Bagging	1.0000	0.9649	0.0351
AdaBoost	1.0000	0.9737	0.0263
Gradient Boosting	1.0000	0.9649	0.0351
Stacked Ensemble	0.9890	0.9649	0.0241

10 Result Visualization

10.1 Confusion Matrices

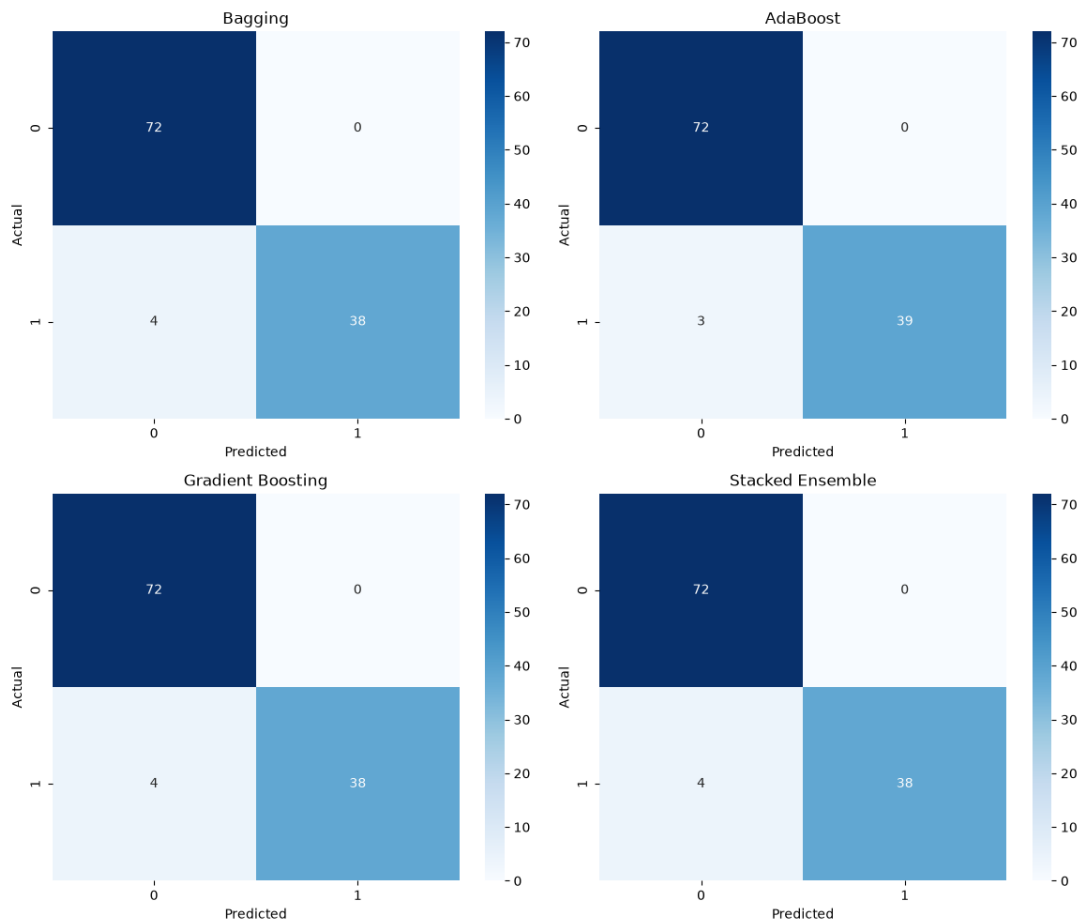


Figure 3: Confusion Matrices for the Four Ensemble Models

Inference: The confusion matrices reveal that all four ensemble classifiers perform exceptionally well on the test set of 114 samples (42 Malignant, 72 Benign). Notably, all models achieve a False Positive (FP) count of exactly 0, yielding a perfect Precision of 1.0000. In a clinical diagnostic setting, this means zero patients are falsely diagnosed with cancer when they have a benign mass.

Regarding False Negatives (FN), which are highly critical as they represent undiagnosed cancer, the tuned AdaBoost model records 3 False Negatives (39 True Positives), outperforming Bagging, Gradient Boosting, and Stacking which each record 4 False Negatives (38 True Positives). This reduction demonstrates boosting's capability to draw sharper decision boundaries near positive class boundaries.

10.2 ROC Curves and AUC

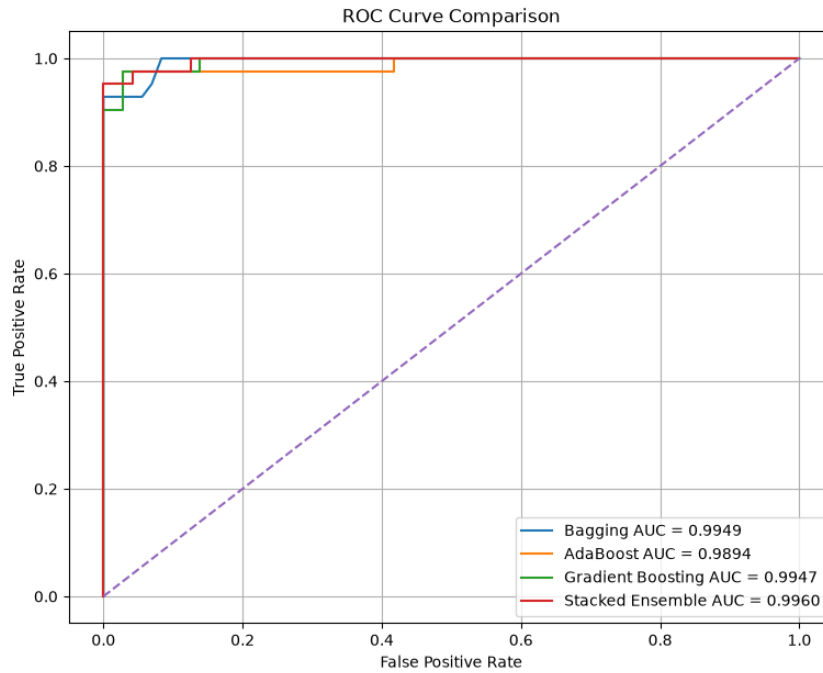


Figure 4: ROC Curve Comparison of Ensemble Classifiers

Inference: The Receiver Operating Characteristic (ROC) curves illustrate the trade-off between True Positive Rate and False Positive Rate across all classification thresholds. The Stacked Ensemble achieves the highest Area Under the Curve (AUC) of 0.9960, demonstrating superior probability ranking capability. It is followed closely by Bagging (0.9949), Gradient Boosting (0.9947), and AdaBoost (0.9894). Stacking obtains a higher AUC because it combines the probabilistic confidence distributions of SVM, Naive Bayes, and Decision Tree. This suggests that while AdaBoost is superior for default hard classification splits, Stacking provides the most robust probability rankings for clinical threshold adjustments.

11 Discussion

11.1 Best-Performing Classifier

Based on the test set evaluations (Table 7) and cross-validation performance, the best-performing model depends on the metric of interest:

1. **AdaBoost** achieves the highest classification accuracy of **97.37%** and F_1 -score of **0.9630** on the test set. It also achieves the highest average CV Accuracy (**97.58%**) and CV F_1 -score (**0.9672**).
2. **Stacked Ensemble** achieves the highest ROC-AUC value of **0.9960** on the test set, indicating superior diagnostic probability ranking. It also exhibits the smallest generalization gap (**0.0241**) and trains rapidly (**0.1120s**), making it the most robust model against overfitting.

11.2 Observation Questions

1. How does Bagging reduce variance?

Bagging (Bootstrap Aggregation) reduces variance by training multiple base models (e.g., Decision Trees) independently on different bootstrap samples generated with replacement from the training set. Because each tree is trained on a slightly different subset of samples and features (0.5 feature ratio in our best model), their individual errors and overfitting behaviors are highly de-correlated. When their predictions are averaged (or voted upon), the uncorrelated high-frequency variance of individual trees cancels out, leaving the ensemble's overall bias unchanged but significantly reducing prediction variance.

2. How does Boosting address model bias?

Boosting addresses bias by training weak learners sequentially. In each iteration, the model evaluates training classification errors and updates sample weights, increasing the weights of misclassified instances. This forces the next weak learner in the sequence to focus on hard-to-classify samples. By taking a weighted combination of these sequentially generated weak learners, boosting constructs a highly complex non-linear decision boundary, thereby converting high-bias base models (like depth-2 stumps) into a low-bias ensemble.

3. Why does stacking benefit from heterogeneous models?

Stacking benefits from heterogeneous base models (e.g., SVM, Naive Bayes, Decision Trees) because they represent fundamentally different mathematical paradigms. SVM searches for maximum-margin geometric hyperplanes, Naive Bayes models conditional probabilities using feature density assumptions, and Decision Trees recursively partition feature spaces using axis-aligned thresholds. Because their underlying decision structures differ, they make distinct, uncorrelated classification errors. The meta-learner (Logistic Regression) learns these patterns and learns how to weight their outputs to correct their individual diagnostic weaknesses.

Which ensemble method performed best and why?

The AdaBoost model achieved the best hard classification results, recording a test accuracy of **97.37%** and F1-score of **0.9630**. It performed best because sequential weight adjustments enabled it to fit the boundaries of the WDBC dataset closely, minimizing False Negatives to 3. However, Stacked Ensemble achieved the best ranking capability, yielding a near-perfect ROC-AUC of **0.9960** and the smallest generalization gap (**2.41%**). Stacking performed best in terms of generalization because it aggregated the probabilistic predictions of diverse classifiers, preventing any single model's bias from dominating and smoothing the decision boundaries.

12 Conclusion

In this experiment, Bagging, Boosting (AdaBoost and Gradient Boosting), and Stacked Ensemble classifiers were implemented, optimized, and compared on the Wisconsin Diagnostic Breast Cancer dataset. Stratified 5-Fold Cross-Validation successfully identified optimal hyperparameters: Bagging with 50 estimators and 0.5 max features; AdaBoost with 150 estimators, 1.0 learning rate, and depth-2 trees.

The final evaluation showed that all models achieved 100% precision (0 False Positives). AdaBoost recorded the highest testing accuracy of **97.37%** and the lowest false

negative rate (3 FN), while Stacked Ensemble achieved the highest ROC-AUC of **0.9960** and the smallest generalization gap (**2.41%**). These findings demonstrate that ensembling successfully addresses the limitations of individual classifiers: Bagging and Stacking mitigate high variance and control overfitting, while Boosting sequentially reduces bias to achieve highly accurate diagnostic boundaries.

13 References

- [1] Scikit-learn Documentation. *Ensemble methods*. <https://scikit-learn.org/stable/modules/ensemble.html>
- [2] Scikit-learn Documentation. *Bagging classifier*. <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.BaggingClassifier.html>
- [3] UCI Machine Learning Repository. *Breast Cancer Wisconsin (Diagnostic) Dataset*. [https://archive.ics.uci.edu/ml/datasets/Breast+Cancer+Wisconsin+\(Diagnostic\)](https://archive.ics.uci.edu/ml/datasets/Breast+Cancer+Wisconsin+(Diagnostic))
- [4] Breiman, L. (1996). *Bagging Predictors*. Machine Learning, 24(2), 123-140.
- [5] Freund, Y., & Schapire, R. E. (1997). *A Decision-Theoretic Generalization of On-Line Learning and an Application to Boosting*. Journal of Computer and System Sciences, 55(1), 119-139.